Instituto Tecnológico de Estudios Superiores de Monterrey
Campus Toluca

Análisis de Algoritmos Avanzados

Análisis Colaborativo: Anomalía Genética

Profesores: Alison Muñoz Capote

Alumnos:

Yael Sebastián González Hernández - A01770992
José Pedro Rodríguez Munguía - A01769631
Marco Aurelio Tamez Robles - A01770750
Ileana Tapia Castillo - A01773374

Para este reto nos repartimos las cuatro técnicas que hemos visto en clase para trabajar con cadenas: Manacher, hashing de cadenas, suffix array y Longest Common Substring. La idea era que cada quien llegara a la reunión de equipo con su técnica ya pensada para este problema en particular, y a partir de ahí discutir cuál convenía más para el juez, considerando que el límite de tiempo es de 1 segundo y el de memoria de 256 MB.

Las cuatro técnicas resuelven el problema de encontrar la subcadena palindrómica más larga, pero no todas llegan ahí de la misma forma. Manacher expande cada centro aprovechando la simetría de lo que ya se calculó, así que nunca vuelve a comparar de más. El hashing convierte el problema en revisar, para cada centro, si dos mitades de la cadena son iguales usando una comparación de hashes en tiempo constante, y luego usa búsqueda binaria sobre el radio del palíndromo. El suffix array ataca el problema por otro lado: construye T = s + "#" + reverso(s) y usa el arreglo LCP para encontrar cuánto coincide un sufijo de s con su espejo en el reverso. Y la variante de Longest Common Substring plantea básicamente lo mismo que el suffix array, pero resuelto con programación dinámica clásica sobre s y su reverso, comparando carácter por carácter en una tabla.

Técnica	**Tiempo**	**Espacio**	**Comentarios del equipo**
Manacher	O(n)	O(n)	Es la única que garantiza tiempo lineal sin importar el caso. No depende de azar ni de construir estructuras auxiliares pesadas; solo hay que manejar bien el caso par/impar.
Hashing de cadenas	O(n log n)	O(n)	Con búsqueda binaria sobre el radio y hashing polinomial queda bastante rápido en la práctica, pero siempre existe la posibilidad de una colisión que arruine un caso de prueba si no se eligen bien el módulo y la base.
Suffix Array + LCP	$O(n \log^2 n)$	O(n)	Funciona (se arma T = s + "#" + reverso(s) y se busca el LCP entre sufijos espejo), pero es la implementación más larga de las cuatro y la constante oculta se nota en la práctica.
Longest Common Substring (DP)	$O(n^2)$	$O(n^2)$	Es la más intuitiva de plantear (comparar s contra su reverso), pero tanto el tiempo como la memoria crecen al cuadrado, así que para las cadenas más grandes del juez simplemente no cabe en 256 MB ni en 1 segundo.

Al final decidimos programar la solución con el algoritmo de Manacher. La razón principal es el límite de tiempo: como es O(n), es la única de las cuatro que nos deja tranquilos sin importar qué tan grande venga la cadena en los casos de prueba del juez; las otras tres, aunque funcionan, tienen un margen mucho más apretado o de plano se salen del límite en el peor caso. La versión

de Longest Common Substring con programación dinámica fue la primera que descartamos, porque tanto el tiempo como la memoria crecen al cuadrado, y con una cadena grande eso truena antes de llegar al segundo límite de 256 MB, no solo al de tiempo.

El hashing nos gustó porque es rápido de programar y en la práctica corre muy bien, pero no nos convenció del todo por el riesgo de colisiones: si el módulo o la base del polinomio no se eligen con cuidado, existe la posibilidad de que el juez tenga un caso diseñado para tronar ese hash, y no queríamos apostarle a eso en una entrega que solo se manda una vez. El suffix array con LCP sí es correcto y en teoría tiene una complejidad razonable, pero terminó siendo la implementación más larga y con más piezas que podían salir mal (construir el suffix array, el LCP con Kasai, la sparse table para las consultas de rango), y para el tamaño de cadena que maneja este problema no nos daba ninguna ventaja real sobre Manacher que justificara ese riesgo adicional.

En resumen, Manacher fue la opción que mejor equilibró simplicidad de implementación, certeza de que no habrá TLE ni problemas de memoria, y ausencia de casos raros que puedan fallar por azar. Las otras tres técnicas quedaron documentadas y discutidas en equipo, pero es la que finalmente se entrega programada en solucion.py para el juez.